

Software Engineering Practice Final

1. Consider the following code. Draw a picture of what the stack and heap would look like just after executing line 18 (but before returning from the method):

```
5- public static class SimpleList {  
6     String[] arrayOfStrings = new String[3];  
7     int arrIndex = 0;  
8  
9-     public void add(String s) {  
10         arrayOfStrings[arrIndex] = s;  
11         arrIndex++;  
12     }  
13 }  
14  
15- public void doAThing() {  
16     String s = "Hello World";  
17     SimpleList list = new SimpleList();  
18     list.add(s);  
19 }
```

2. Suppose you have the following code:

```
public static interface Calculator {
    int doCalculation(int a, int b);
}

public static class SlowCalculator implements Calculator {
    @Override
    public int doCalculation(int a, int b) {
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        return a + b;
    }
}

public static class FastCalculator implements Calculator {
    @Override
    public int doCalculation(int a, int b) {
        return a + b;
    }
}
```

Write a benchmark test to verify that FastCalculator is faster than SlowCalculator, that will print how much faster it is.

3. Consider the following class:

```

public static class DataMessage {
    private final int key;
    private final String value;

    public DataMessage(int key, String value) {
        this.key = key;
        this.value = value;
    }

    public int getKey() {
        return key;
    }

    public String getValue() {
        return value;
    }
}

```

Write the .proto code to define a message that represents this class

4. Consider the following code. Line 26 requires an explicit cast to Double to make the code as written compile, but it's not great from a type-safety perspective. How could you change the code so that the cast is no longer necessary?

```

12 public static interface CallableCreator {
13     Callable<Double> makeCalculatorThread(int val);
14 }
15
16 public double doAThing(CallableCreator creator) throws Exception {
17     ExecutorService threadPool = Executors.newCachedThreadPool();
18     List<Future<?>> futures = new ArrayList<>();
19
20     for (int i = 0; i < 10; i++) {
21         futures.add(threadPool.submit(creator.makeCalculatorThread(i)));
22     }
23
24     double result = 0;
25     for (Future<?> f : futures) {
26         result += (Double)f.get();
27     }
28     return result;
29 }

```

5. Imagine you were writing a program that included a `LibraryReferenceDesk` and a `LibraryCatalog` component. For now, the implementation of the catalog is a simple in-memory storage. Write a test that will verify the **interaction between** `LibraryReferenceDeskImpl` and `LibraryCatalogImpl`, assuming the following interfaces and classes:

```
public interface LibraryCatalog {
    // returns a list of all book titles in the catalog
    List<String> getAllBookTitles();

    // adds a book with the specified title to the catalog
    void addBookToCatalog(String bookTitle);
}

public interface LibraryReferenceDesk {
    // Returns true if a book with that title is in the catalog
    boolean hasBook(String title);
}

public class LibraryCatalogImpl implements LibraryCatalog {
    List<String> bookTitles = new ArrayList<>();

    @Override
    public List<String> getAllBookTitles() {
        return bookTitles;
    }

    @Override
    public void addBookToCatalog(String bookTitle) {
        bookTitles.add(bookTitle);
    }
}

public class LibraryReferenceDeskImpl implements
LibraryReferenceDesk {
```

```
private LibraryCatalog catalog;

public LibraryReferenceDeskImpl(LibraryCatalog catalog) {
    this.catalog = catalog;
}

@Override
public boolean hasBook(String title) {
    return catalog.getAllBookTitles().contains(title);
}
}
```

6. (a) What's a scenario where you would prefer a multi-process solution to a multi-threaded solution?

(b) What's a scenario where you would prefer a multi-threaded solution to a multi-process solution?

7. Suppose you have the following client and server code:

Client.java:

```
public void add() {
    AddingRequest request =
AddingRequest.newBuilder().setValue1(2).setValue2(3).build();
    AddingResponse response = blockingStub.addNumbers(request);

    if (response.hasErrorMessage()) {
        System.err.println("Error: " + response.getErrorMessage());
    } else {
        System.out.println("Sum: " + response.getSum());
    }
}
```

Server.java:

```
public void addNumbers(AddingRequest request,
    StreamObserver<AddingResponse> responseObserver) {
    int value1 = request.getValue1();
    int value2 = request.getValue2();
    AddingResponse result;
    try {
        result = AddingResponse.newBuilder().setSum(value1 +
value2).build();
    } catch (Exception e) {
        result = AddingResponse.newBuilder()
            .setErrorMessage("Server error: " +
e.getMessage())
            .build();
    }

    responseObserver.onNext(result);
    responseObserver.onCompleted();
}
```

When you run the server and client, you get the following printed on the console for Client.java:

Error: Server error: NullPointerException

What file/method would you put a breakpoint on to start diagnosing the issue?

8. What's a situation where you'd prefer to use a `Semaphore` rather than a `synchronized` block?